HUNARMAND PUNJAB

FINAL PROJECT REPORT

LearnHub

A MERN Stack Learning Management System

Course	MERN Stack Web Development
Project	LearnHub – Learning Management System
Batch	3
Total Marks	40
Student Name	Maryam Shaikh
Registration No.	HM-B3-S2-2026-160233
Instructor	Muhammad Noor
Submission Date	11-November-2026
GitHub repository link.	https://github.com/MaryamShaikh124/learn_hub
live deployment link	https://learnhub-frontend-wheat.vercel.app/

Submitted as the final project for MERN Stack Web Development.

Project overview

LearnHub is a web-based Learning Management System (LMS) developed to bring students, instructors and administrators into one platform. Students can discover courses and access learning activities, instructors can manage course content, and administrators can oversee platform users and activity. The application follows the MERN stack architecture: MongoDB for data storage, Express and Node.js for backend services, and React for the user interface.

Project objectives

• Build a responsive React application with clear navigation and reusable interface components.

• Develop REST API endpoints using Node.js and Express.

• Store application data in MongoDB using Mongoose models.

• Implement user authentication using JSON Web Tokens (JWT) and password hashing with bcrypt.

• Apply role-based access for Admin, Instructor and Student accounts.

• Support course discovery, course management and student enrollment.

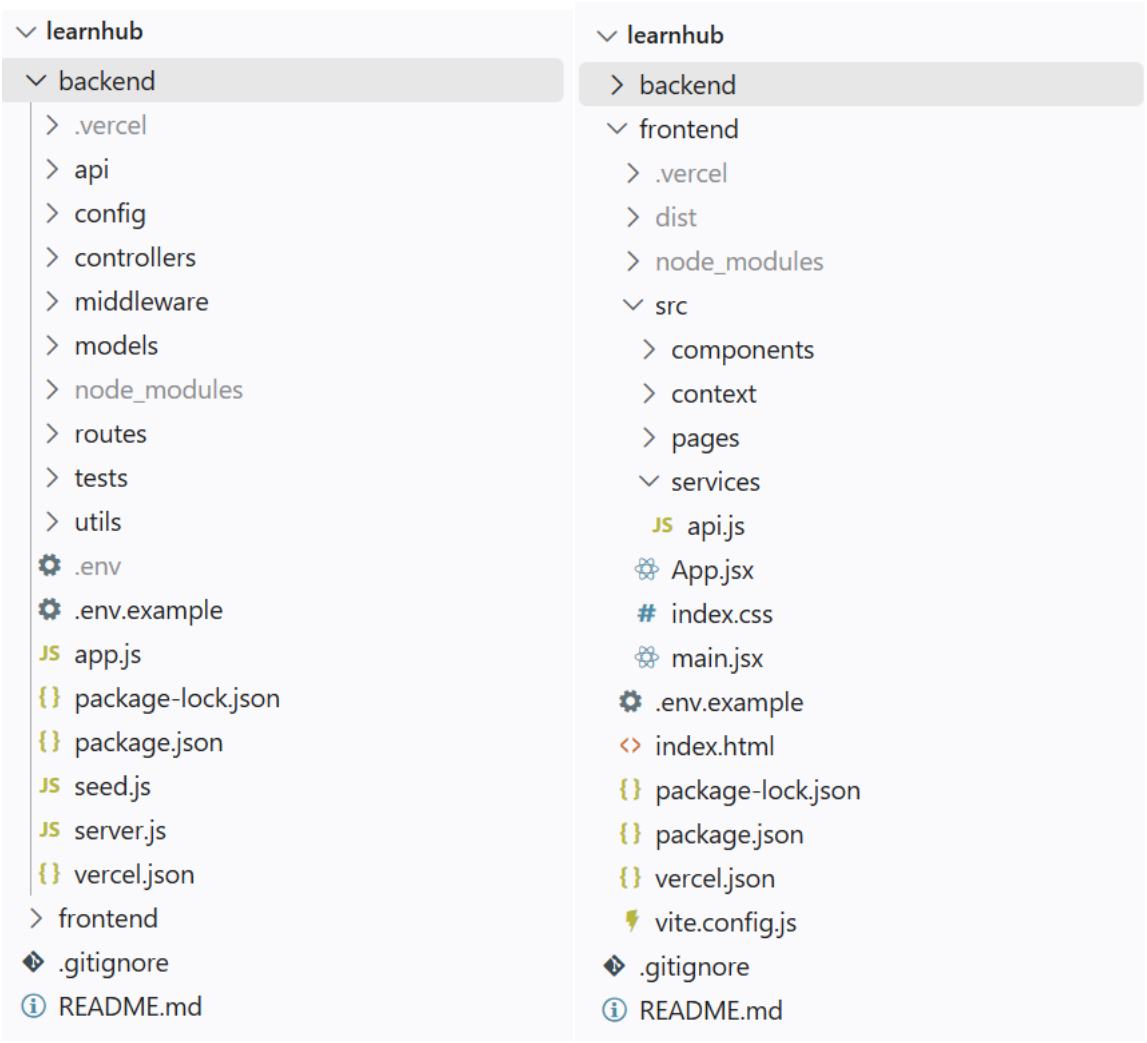
• Prepare the frontend and backend for cloud deployment and test their integration.

1. Technologies and tools

Layer	Technology / Tool	Purpose
Frontend	React, React Router	Pages, reusable components and navigation
HTTP client	Axios	Frontend-to-backend API communication
Backend	Node.js, Express	REST API and application logic
Database	MongoDB Atlas, Mongoose	Persistent data storage and schemas
Authentication	JWT, bcryptjs	Token-based authentication and password hashing
Security / middleware	Helmet, CORS, express-rate-limit	Security headers, cross-origin access and request limiting
Deployment	Vercel	Hosting frontend and backend
Development	VS Code, Git, GitHub	Coding, version control and source hosting

2. Project structure

The project is organized into separate frontend and backend directories. The backend uses dedicated folders for configuration, controllers, middleware, models and routes. The frontend separates application pages, components, routing and API services.

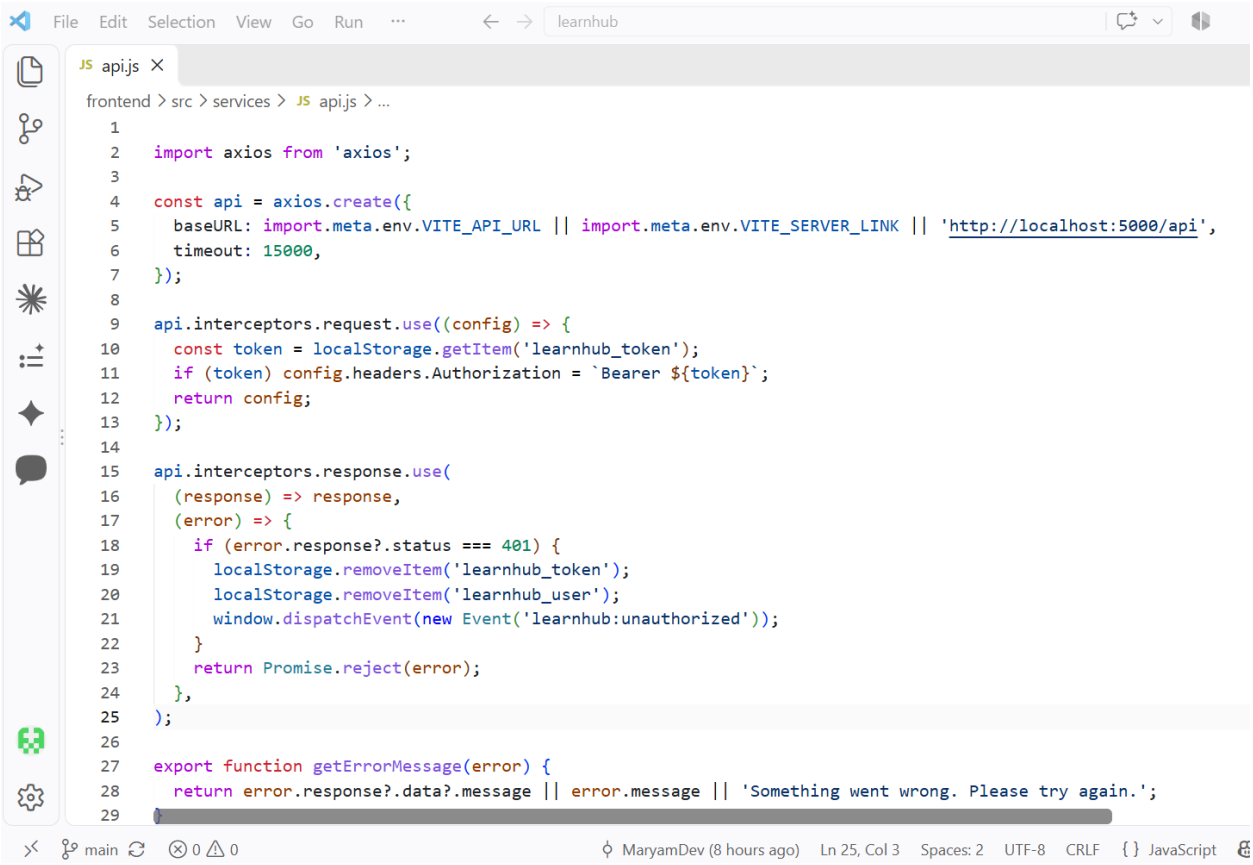


3. Frontend implementation

The frontend is built with React. React Router is used to navigate between application pages. React state and effect hooks support interactive elements and data loading. Lists of courses and other records are rendered dynamically from API responses, while conditional rendering is used for loading states, errors and role-specific content.

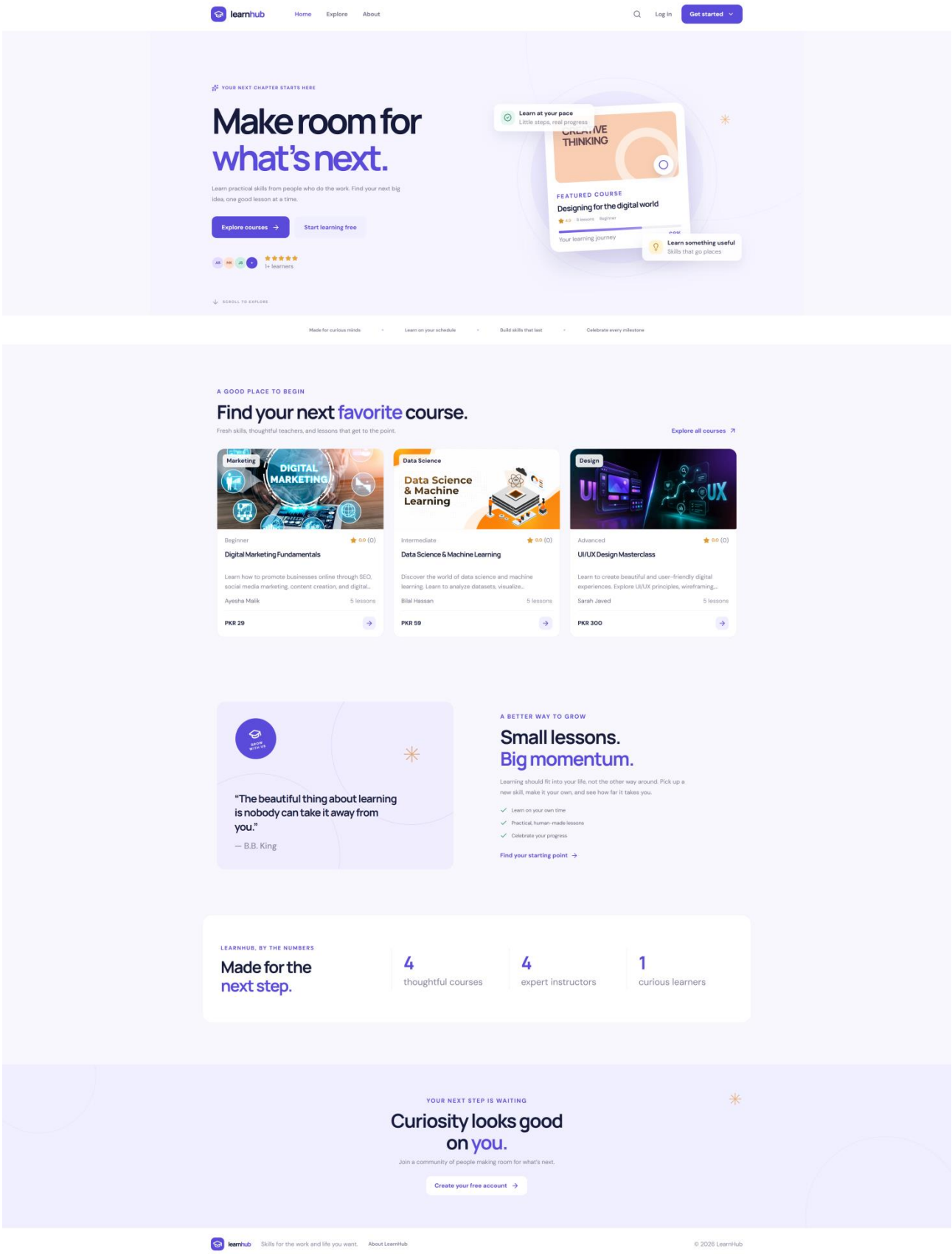
The Axios service centralizes communication with the backend. The API base URL is read from the Vite environment variable VITE_SERVER_LINK, allowing the deployed API URL to be configured without hard-coding it into components. A request interceptor attaches the stored JWT as a Bearer token when available. A response interceptor handles unauthorized responses by clearing local authentication data and notifying the application.

Frontend API service — important code section

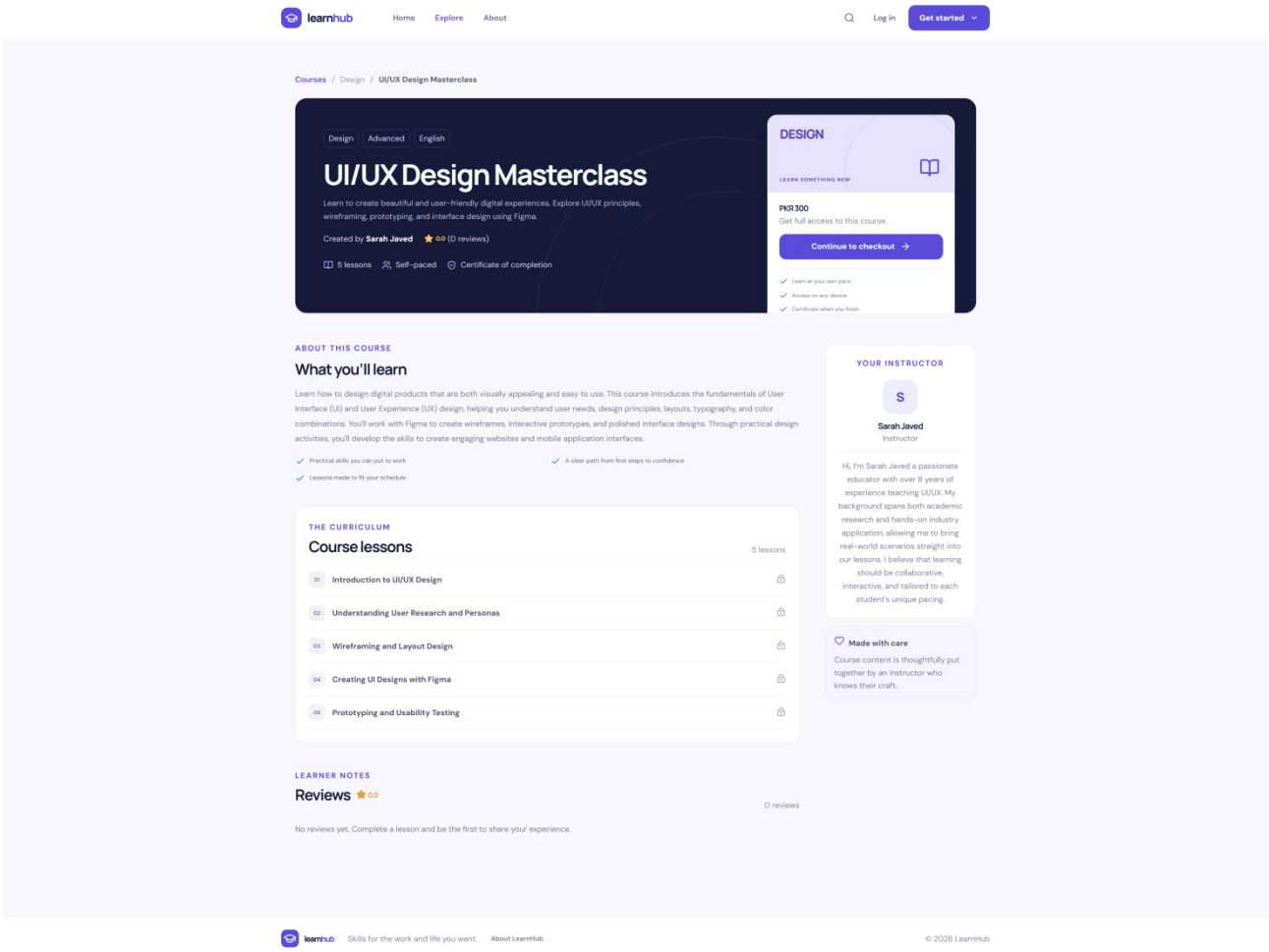


Core pages and interface evidence

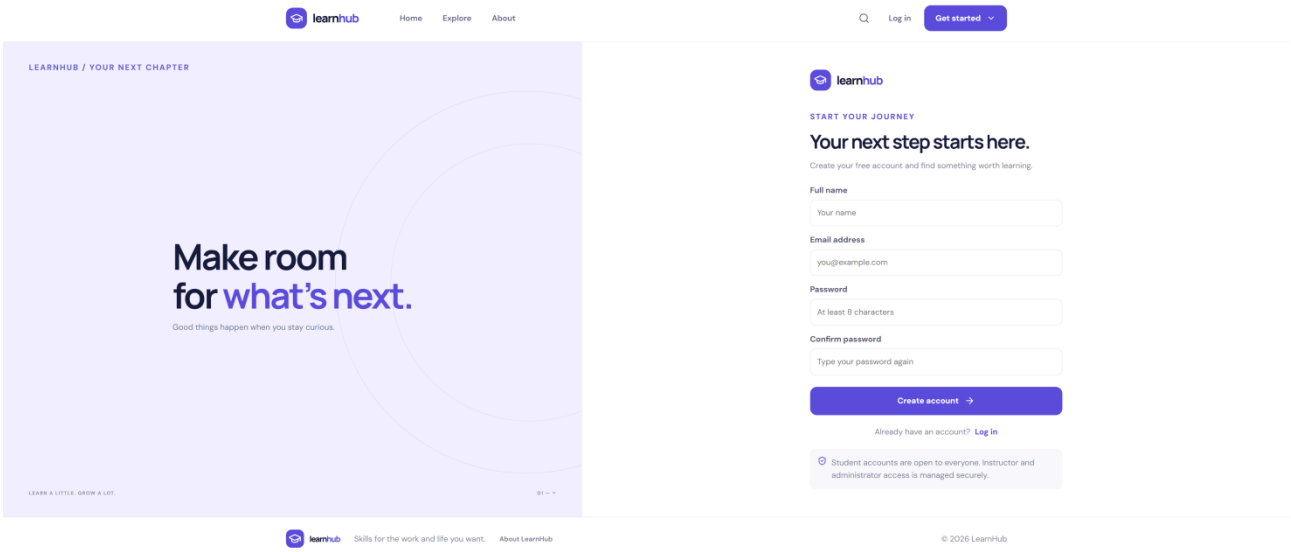
Home Page:



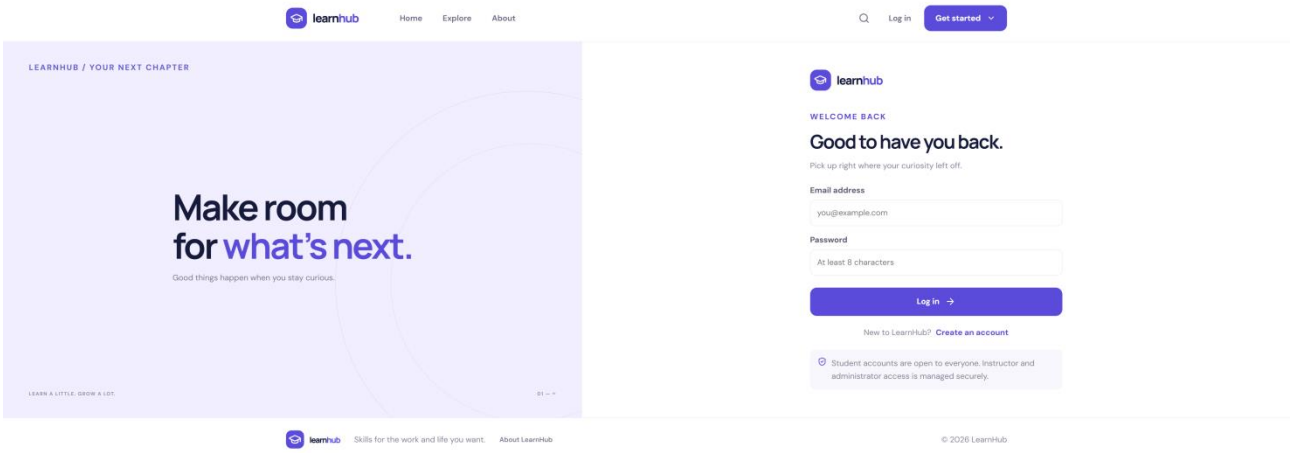
Course Listing and course detail:



Registration

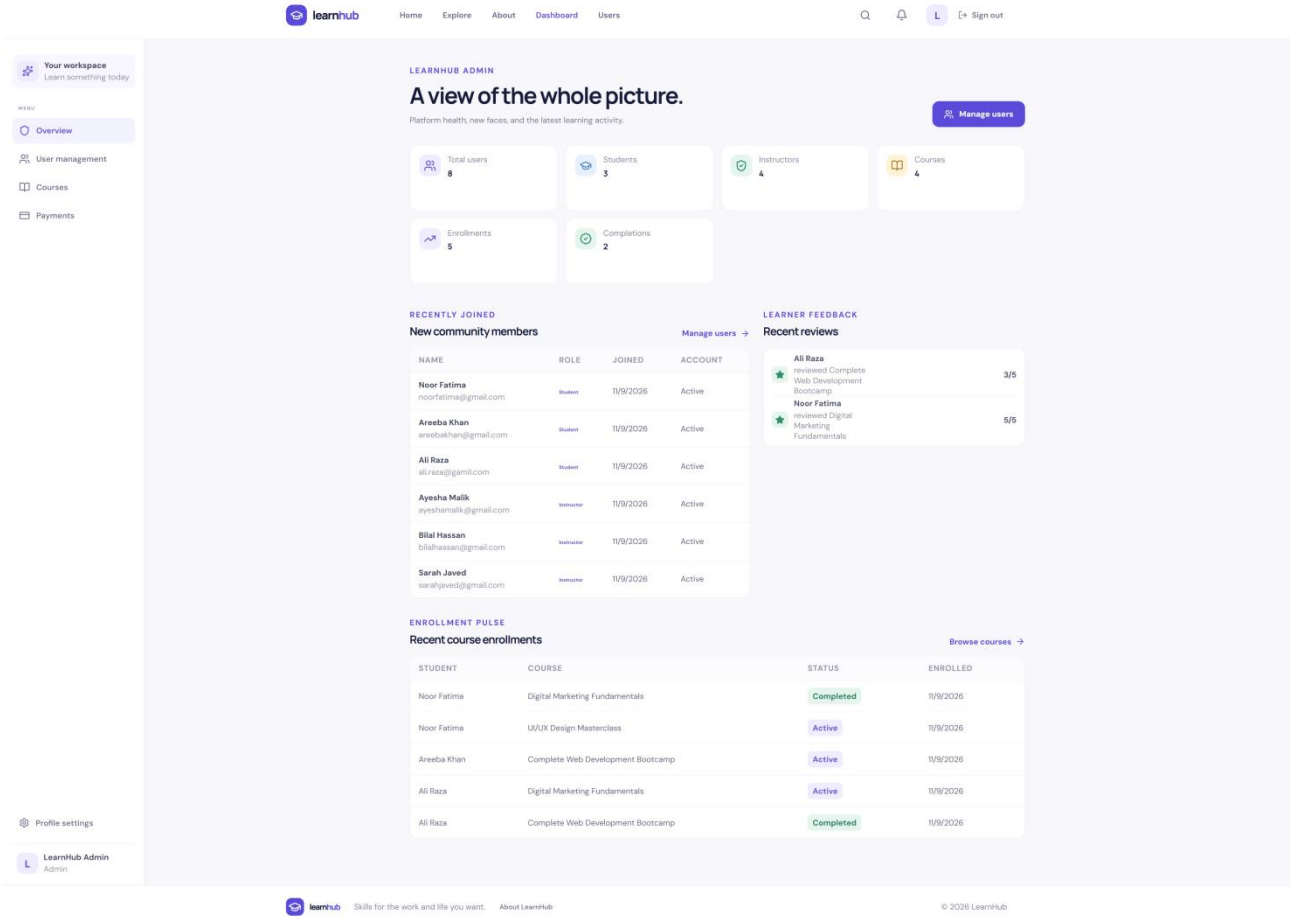


Login:

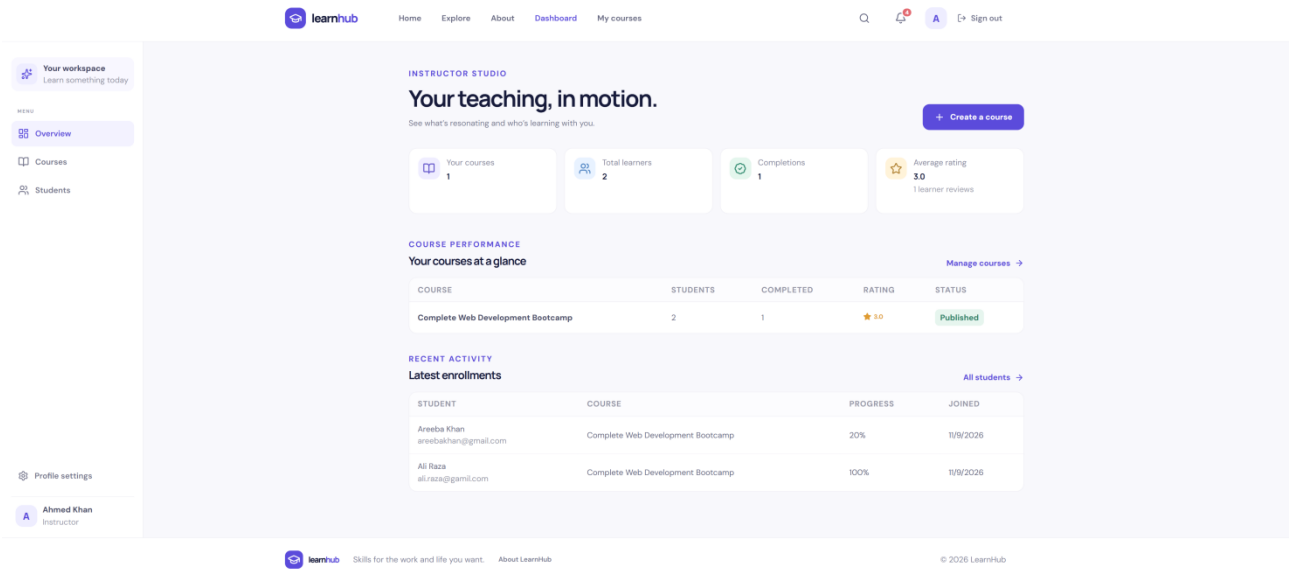


Role-based dashboards:

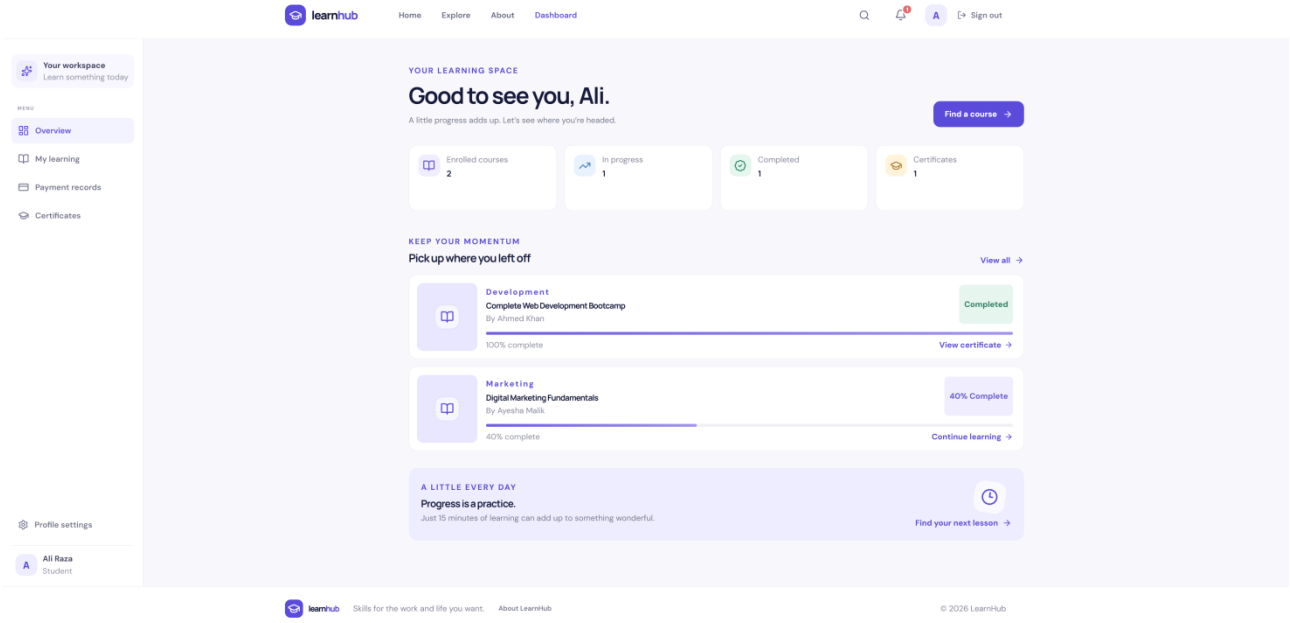
Admin:



Instructor:


learnhub
Home
Explore
About
Dashboard
My courses
Sign out
Your workspace
Learn something today
MENU
Overview
Courses
Students
Profile settings
Ahmed Khan
Instructor
INSTRUCTOR STUDIO
Your teaching, in motion.
See what's resonating and who's learning with you.
Create a course
Your courses
1
Total learners
2
Completions
1
Average rating
3.0
1 learner reviews
COURSE PERFORMANCE
Your courses at a glance
Manage courses
COURSE
STUDENTS
COMPLETED
RATING
STATUS
Complete Web Development Bootcamp
2
1
3.0
Published
RECENT ACTIVITY
Latest enrollments
All students
STUDENT
COURSE
PROGRESS
JOINED
Areeba Khan
areebakhan@gmail.com
Complete Web Development Bootcamp
20%
11/9/2026
Ali Raza
ali.raza@gamil.com
Complete Web Development Bootcamp
100%
11/9/2026
learnhub
Skills for the work and life you want.
About LearnHub
© 2026 LearnHub

Student:


learnhub
Home
Explore
About
Dashboard
Sign out
Your workspace
Learn something today
MENU
Overview
My learning
Payment records
Certificates
Profile settings
Ali Raza
Student
YOUR LEARNING SPACE
Good to see you, Ali.
A little progress adds up. Let's see where you're headed.
Find a course
Enrolled courses
2
In progress
1
Completed
1
Certificates
1
KEEP YOUR MOMENTUM
Pick up where you left off
View all
Development
Complete Web Development Bootcamp
By Ahmed Khan
Completed
100% complete
View certificate
Marketing
Digital Marketing Fundamentals
By Ayesha Malik
40% Complete
40% complete
Continue learning
A LITTLE EVERY DAY
Progress is a practice.
Just 15 minutes of learning can add up to something wonderful.
Find your next lesson
learnhub
Skills for the work and life you want.
About LearnHub
© 2026 LearnHub

4. Backend and database implementation

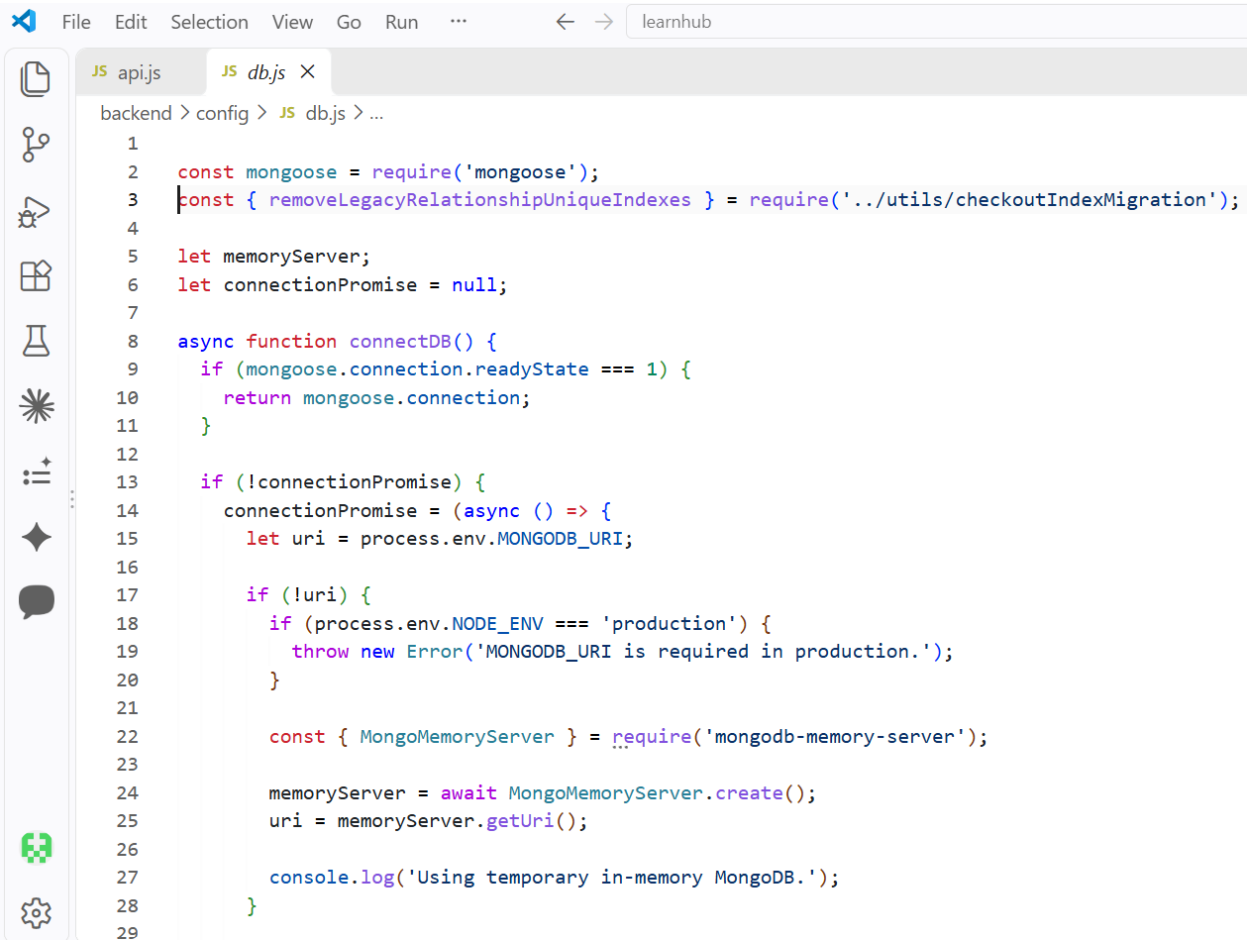
The backend is built using Node.js and Express. Express routes direct requests to the appropriate controllers. Middleware is used for security headers, CORS, request logging, JSON parsing, rate limiting, authentication and error handling. Mongoose provides schema-based interaction with MongoDB.

The shared Express application defines the API route groups. The following route mounts are present in the supplied app.js file:

Route mount	Purpose
/api/health	Health check
/api/stats	Public statistics
/api/auth	Authentication routes
/api/courses	Course routes
/api	Lesson routes
/api/enrollments	Enrollment routes
/api/reviews	Review routes
/api/certificates	Certificate routes
/api/notifications	Notification routes
/api/instructor	Instructor routes
/api/admin	Admin routes

Database connection

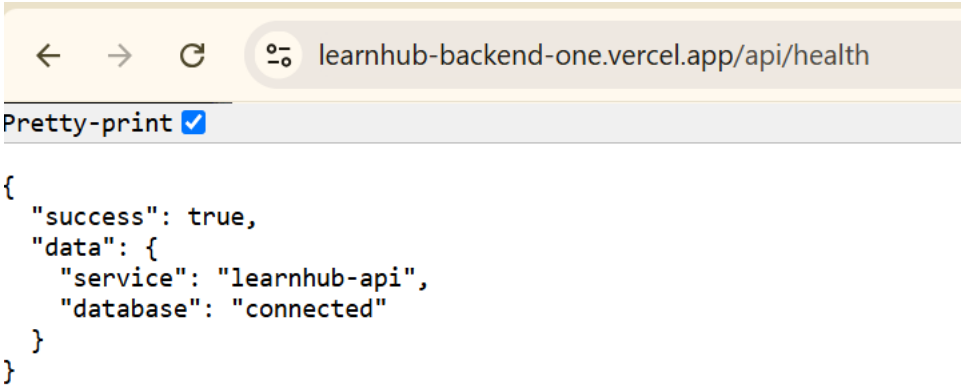
The database configuration reads the MongoDB connection string from the MONGODB_URI environment variable. In production, the application throws an error if this value is missing. The connection helper reuses an existing Mongoose connection and shares an in-progress connection promise to reduce repeated connection attempts in a serverless runtime.



Express application and CORS

The Express application enables CORS before mounting API routes. The allowed client origins are read from CLIENT_URL, which can contain a comma-separated list of origins. This is necessary because the browser restricts cross-origin requests unless the backend explicitly allows the requesting frontend origin.

Important configuration: CLIENT_URL must match the deployed frontend origin exactly, including the current production domain, and must not include an API path.



5. Authentication and role-based authorization

LearnHub supports three user roles: Student, Instructor and Admin. JWT-based authentication is used to identify authenticated users, while bcryptjs is used to hash passwords rather than storing plain-text passwords. Protected backend routes use authentication and role authorization middleware. Frontend route protection is used to direct users to the appropriate dashboard and restrict role-specific pages.

Role responsibilities

Role	Main responsibilities
Student	Browse courses, enroll, view enrolled courses and learning progress.
Instructor	Create and manage courses, lessons and course-related information.
Admin	Manage platform users and courses and view administrative analytics.

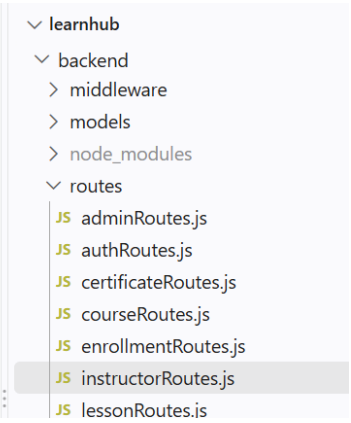
Admin Routes:



Instructor Routes:

backend > routes > JS instructorRoutes.js > ...

```
1   const express = require('express');
2   const { protect, authorize } = require('../middleware/auth');
3   const instructor = require('../controllers/instructorController');
4
5   const router = express.Router();
6
7   router.use(protect, authorize('instructor'));
8   router.get('/analytics', instructor.analytics);
9   router.get('/students', instructor.students);
10
11  module.exports = router;
12
```



6. Course management and enrollment

The course management module provides API routes for course operations. Students can browse course information and use the enrollment functionality. Enrollment records associate a student with a course and can track learning progress. The project also includes route groups for lessons, reviews, certificates and notifications.

Expected end-to-end interaction

• The frontend requests the course list from the backend.

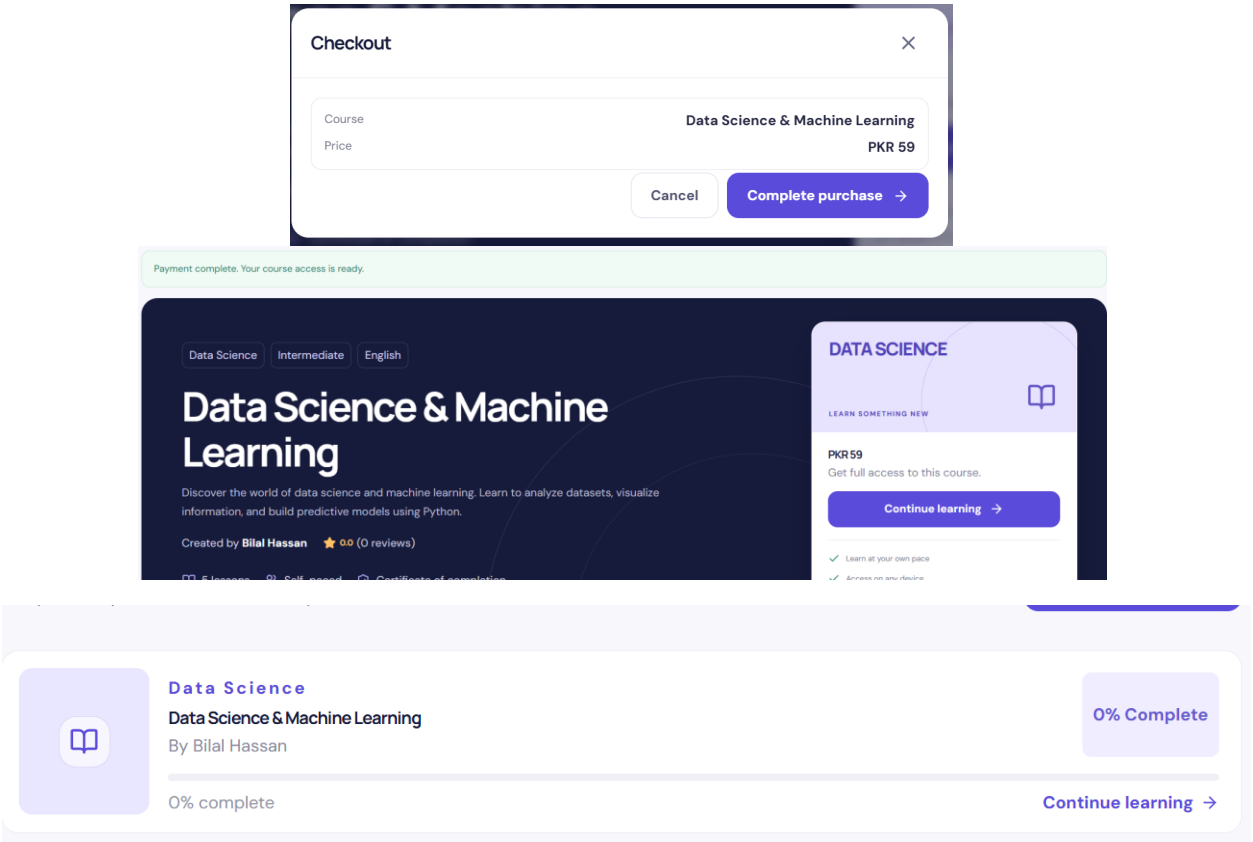
• The backend retrieves course records from MongoDB through Mongoose.

• The frontend displays the returned course data.

• An authenticated student submits an enrollment request.

• The backend validates the request and stores the enrollment.

• The student dashboard requests and displays the student's enrolled courses.

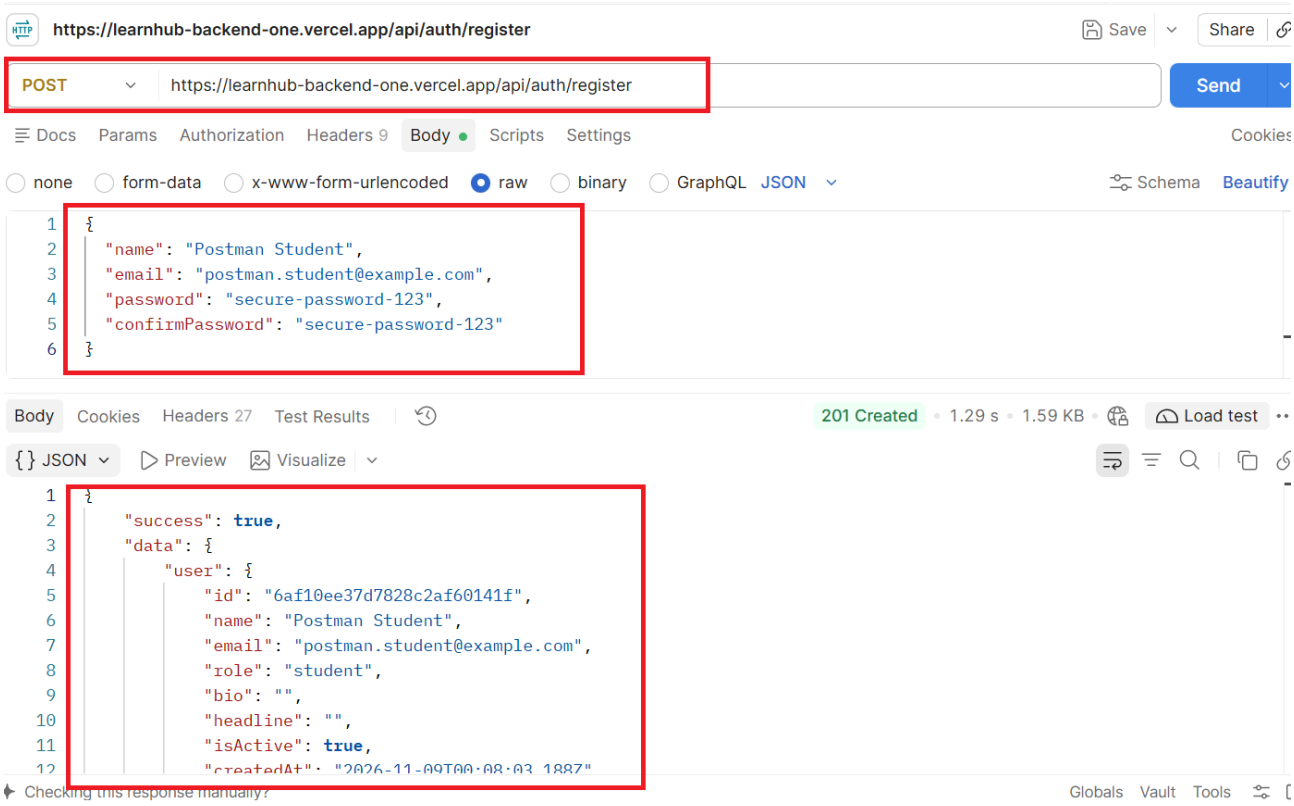


7. API testing and evidence

API testing should demonstrate successful responses as well as validation and authorization behaviour. The following evidence table lists the key requests relevant to the assignment. Use the actual endpoint paths and methods configured in the project when capturing Postman or Thunder Client results.

Test	Method / endpoint	Evidence to capture
Health check	GET /api/health	Successful service and database response
Course listing	GET /api/courses	Course list response
Registration	POST /api/auth/register	New user response; hide private fields

Login	POST /api/auth/login	Authentication response/token; redact token in report
Course create/update/delete	POST, PUT, DELETE course routes	Authorized CRUD response
Enrollment	POST /api/enrollments (verify actual route)	Enrollment creation response
My courses	GET /api/enrollments/my-courses (verify actual route)	Student's enrolled courses



8. Deployment and integration testing

The frontend and backend have been prepared for deployment on Vercel. The backend is configured as an Express serverless function and uses MongoDB Atlas for persistent database storage. The frontend API base URL is configured through the Vite environment variable VITE_SERVER_LINK. The backend CORS configuration reads the permitted frontend origin from CLIENT_URL.

Deployment URLs shared during development

Frontend: https://learnhub-frontend-wheat.vercel.app

Backend: https://learnhub-backend-one.vercel.app

A backend health-check request returned the following response during testing:


```
"success":true,"data":{"service":"learnhub-api","database":"connected"}}
```


Integration issue observed: Browser console evidence showed CORS errors for requests to /api/courses, /api/stats and /api/auth/register. The frontend deployment origin had changed, while the backend's CLIENT_URL needed to be updated to the current frontend origin. The backend environment variable and a fresh backend deployment must be verified before claiming successful end-to-end frontend integration.

9. Error handling and security

• Passwords are hashed using bcryptjs before storage.

• JWT secrets and database credentials are stored in backend environment variables.

• Helmet is used to set security-related HTTP headers.

• CORS restricts browser access to configured frontend origins.

• Express rate limiting is applied to authentication routes.

• A centralized not-found handler and error handler are registered after API routes.

• The frontend handles unauthorized responses by clearing stored authentication data and dispatching an application event.

10. Testing summary

Area	Evidence available / status
Backend deployment	Health endpoint responded successfully.
MongoDB integration	Health response reported database: connected.
Frontend API integration	final successful integration.
Authentication	Capture successful register/login tests before submission.
Role-based access	Capture each role dashboard and a protected-route test.
Course CRUD and enrollment	Capture successful API and browser tests before submission.

11. Conclusion

LearnHub demonstrates the structure and technologies of a MERN Stack Learning Management System. It separates the React frontend from the Express backend, uses MongoDB through Mongoose, centralizes API communication through Axios, and includes authentication, role-oriented functionality, course management and enrollment-related modules. The backend deployment and database health check were confirmed during development. The remaining submission step is to verify and capture successful end-to-end browser/API flows after updating the backend CORS allow-list for the current frontend deployment, then replace the evidence placeholders with genuine screenshots.